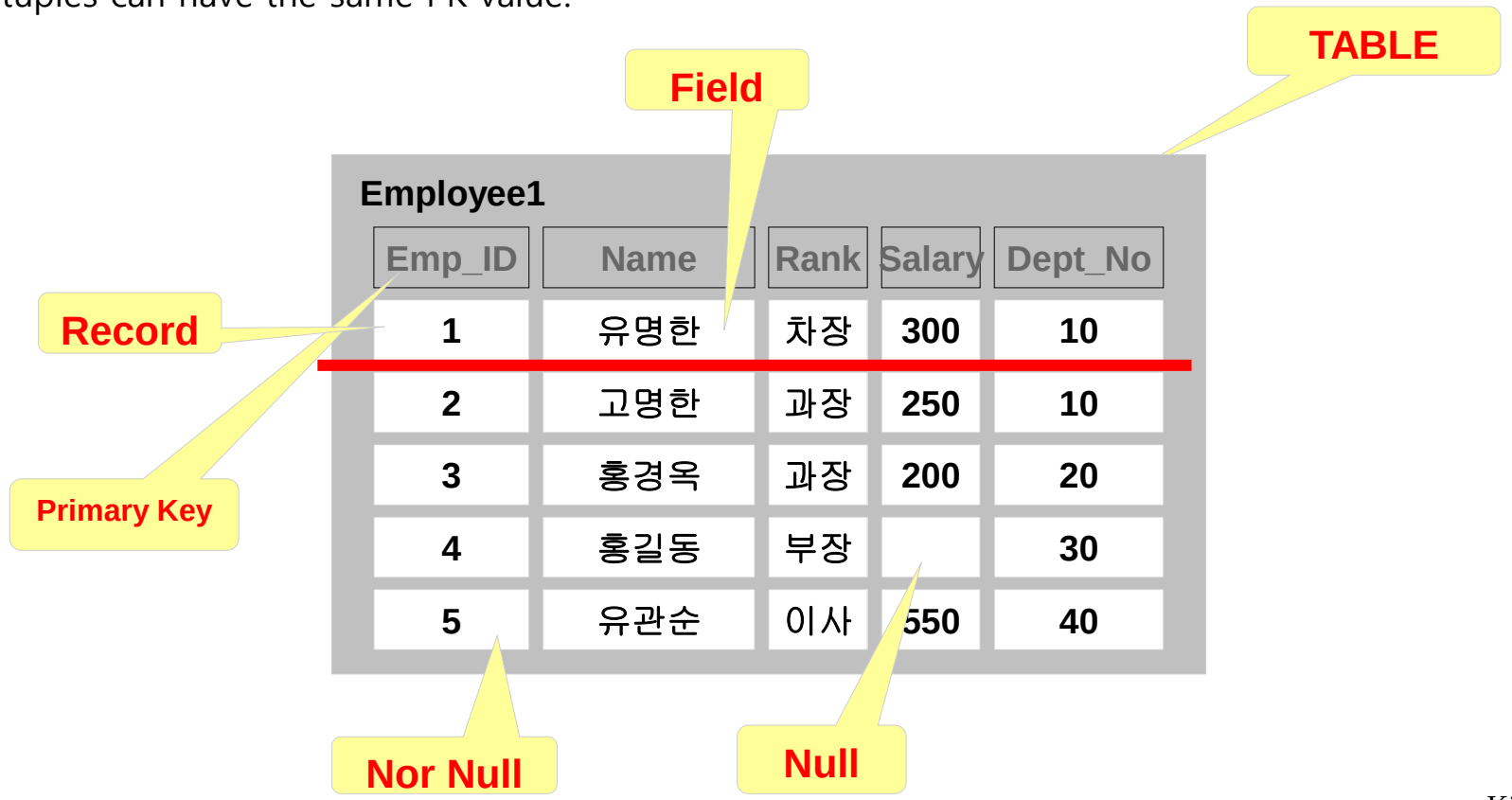
Intro to Relation (Table) and Structured Query Language

Nam, Ki-Hwan
Korea University
(namkh@korea.ac.kr)

Relation Terminology: Informally Table

- Relation: table,
- Record: row,
- Attribute: Field, column
- Primary key:
  - A special attribute, or combination of attributes.
  - No two tuples can have the same PK value.



Employee1

Emp_ID	Name	Rank	Salary	Dept_No
1	유명한	차장	300	10
2	고명한	과장	250	10
3	홍경옥	과장	200	20
4	홍길동	부장		30
5	유관순	이사	550	40

Relational Table

Another Table – Purchase

- Table = set of Fields (“**Pruchase ID**”, “Customer ID”, “Purchase Date”, “ShopID”, “Amount”).
- Individual Purchase Data = Data Record.
- Types of Fields : (1) Identifier Key, (2) Fact (or Score), (3) Dimension

Purchase Table

Field

Data record

구매 ID	고객ID	구매 일자	매장 정보	구매 금액
M10532	K0001	2009/01/01	명동1	20,000
D00583	K0001	2009/03/11	동대문	30,000
K20954	K0001	2009/07/21	강남역2	15,000
K30983	K0002	2009/05/05	김포공항	70,000
Y00398	K0002	2009/05/09	명동2	35,000

Relational Table

Third Table – Customer

- Customer = Fields ("**Customer ID**", "Customer Name", "Gender", "BirthDate").
- Product = (**ProductID**, ProductName, Cost)
- Types of Fields : (1)Primary Key, (2) Foreign Key, (3) Dimension, (4)Fact (or Score),

Customer

고객ID	고객이름	성별	생년월일
K0001	강호동	M	1965/02/08
K0002	김태희	F	1982/03/11
K0003	유재석	M	1971/07/21
K0004	하지원	F	1980/05/05
K0005	현빈	M	1981/05/09

Purchase

구매 ID	고객ID	상품 ID	매장 정보	구매 금액
M10532	K0001	P01	명동1	20,000
D00583	K0001	P02	동대문	30,000
K20954	K0001	P01	강남역2	15,000
K30983	K0002	P03	김포공항	70,000
Y00398	K0002	P04	명동2	35,000

Product

상품 ID	상품이름	원가
P01	Pasta	10,000
P02	Steak	12,000
P03	Sushi	46,000
P04	Pizza	20,000

SQL ?

- **Structured Query Language**
- **High level declarative language for relational database**
- **De facto standard in the database world – ANSI, ISO standard (System R, DB2, Oracle, Sybase)**

SQL

Customer

CustomerID	Name	Gender	Age	Address	VIP
1	Baker	M	38	Seoul	Y
2	David	M	41	Seoul	N
3	Park	F	33	Dae-jeon	N
4	John	F	39	Pusan	N
5	Jenny	M	39	Pusan	Y

Product

ProductID	Product-Category	Manu-acturerID	ProductName	IssueDate	Brand	UnitPrice	Color
1	C	1	Handbag	39867	Louis Vuitton	3000000	Brown
11	C	2	Blouse	39874	Nike	50000	Black
12	C	3	Blouse	39909	Polo	200000	White
13	C	2	Skirt	39909	Nike	150000	Black
14	C	3	Skirt	39890	Polo	300000	Red
2	C	4	Handbag	39861	Prada	2500000	Black
3	C	2	Bag	39869	Nike	200000	White

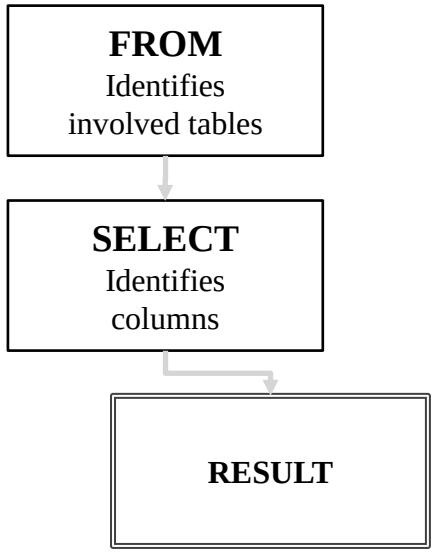
Purchase

Order No	CustomerID	PurchaseDate	PaymentMethod	DeliveryMethod	OrderQuantity	TotalPurchasePrice	ProductID
1	1	39876	Credit Card	shipping	1		1
2	1	39905	Cash	shipping	1		2
3	2	39941	Credit Card	shipping	2		3
4	3	39926	Credit Card	airplane	1		3
5	4	39933	Cash	airplane	1		3
6	3	39934	Credit Card	shipping	1		11
7	1	39937	Credit Card	shipping	1		12
8	2	39936	Cash	airplane	1		13

SQL Structure

SQL Structure: S + F

- SELECT <attribute list>
- FROM <table list>



Example 1)

고객ID	구매금액
K0001	20,000
K0001	30,000
K0001	15,000
K0002	70,000
K0002	35,000

Example 2)

매장정보	구매금액
명동1	20,000
동대문	30,000
강남역2	15,000
김포공항	70,000
명동2	35,000

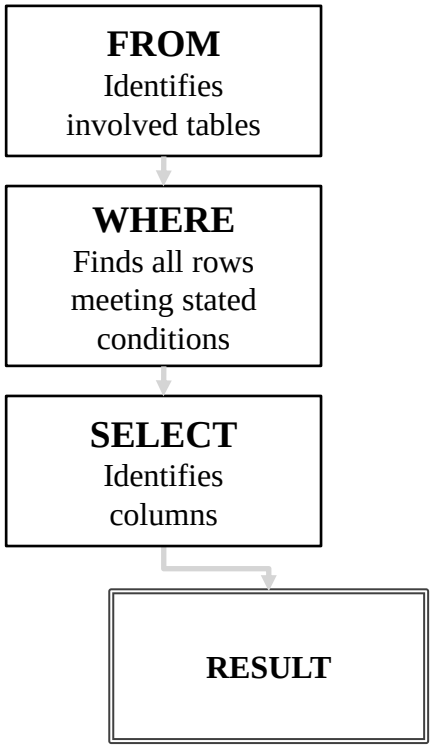
Purchase Table

구매ID	고객ID	구매일자	매장정보	구매금액
M10532	K0001	2009/01/01	명동1	20,000
D00583	K0001	2009/03/11	동대문	30,000
K20954	K0001	2009/07/21	강남역2	15,000
K30983	K0002	2009/05/05	김포공항	70,000
Y00398	K0002	2009/05/09	명동2	35,000

SQL 구조

SQL Structure : S + F + W

- SELECT <attribute list>
- FROM <table list>
- WHERE <conditions>



= : Equal to

> : Greater than

>= : Greater than or equal to

< : Less than

<= : Less than or equal to

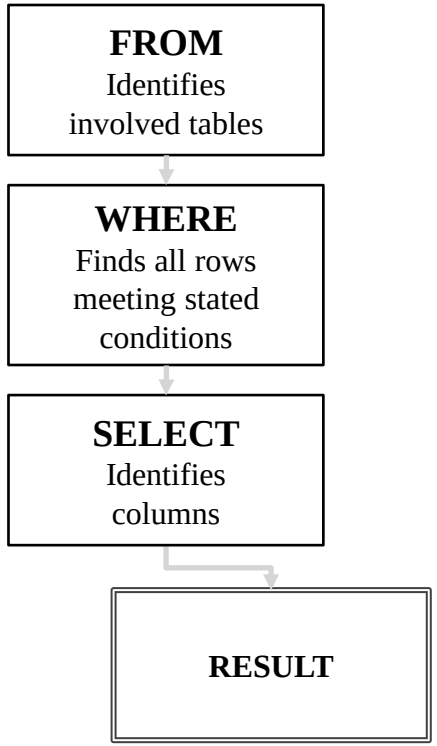
<> : Not equal to

!= : Not equal to

SQL 구조

SQL Structure : S + F + W

- SELECT <attribute list>
- FROM <table list>
- WHERE <conditions>



Example 1)

고객ID	구매금액
~~K0001~~	~~20,000~~
K0001	30,000
~~K0001~~	~~15,000~~
K0002	70,000
K0002	35,000

고객ID	구매금액
K0001	30,000
K0002	70,000
K0002	35,000

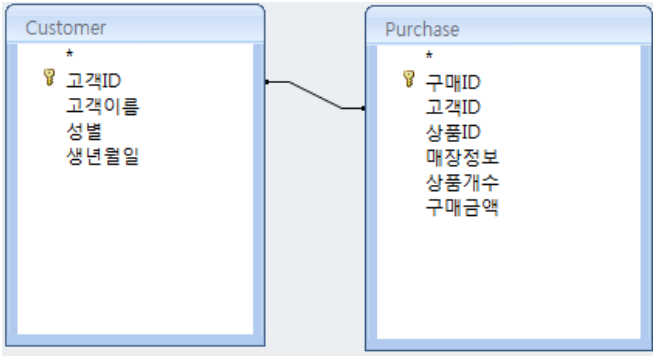
구매테이블

구매ID	고객ID	구매일자	매장정보	구매금액
M10532	K0001	2009/01/01	명동1	20,000
D00583	K0001	2009/03/11	동대문	30,000
K20954	K0001	2009/07/21	강남역2	15,000
K30983	K0002	2009/05/05	김포공항	70,000
Y00398	K0002	2009/05/09	명동2	35,000

SQL 구조

SQL Structure : S + F + W

Example 1) *MS ACCESS



구매ID	Purchase.고객ID	상품ID	매장정보	상품개수	구매금액	Customer.고객ID	고객이름	성별	생년월일
M10532	K0001	P01	명동1	2	30000	K0001	강호동	M	1965-02-08
D00583	K0001	P02	동대문	1	30000	K0001	강호동	M	1965-02-08
K20954	K0001	P01	강남역2	1	15000	K0001	강호동	M	1965-02-08
K30983	K0002	P03	김포공항	4	68000	K0002	김태희	F	1982-03-11
Y00398	K0002	P04	명동2	2	38000	K0002	김태희	F	1982-03-11
K23212	K0003	P05	강남역1	1	12000	K0003	유재석	M	1971-07-21

SQL 구조

SQL Structure : S + F + W

Example 2) *MS ACCESS

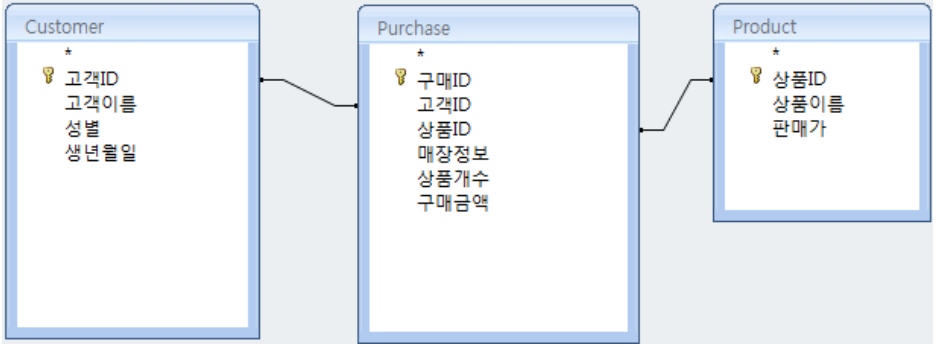
고객이름	성별	구매금액
강호동	M	30000
강호동	M	30000
김태희	F	68000
김태희	F	38000

구매ID	Purchase.고객ID	상품ID	매장정보	상품개수	구매금액	Customer.고객ID	고객이름	성별	생년월일
M10532	K0001	P01	명동1	2	30000	K0001	강호동	M	1965-02-08
D00583	K0001	P02	동대문	1	30000	K0001	강호동	M	1965-02-08
~~K20954~~	~~K0001~~	~~P01~~	~~강남역2~~	~~1~~	~~15000~~	~~K0001~~	~~강호동~~	~~M~~	~~1965-02-08~~
K30983	K0002	P03	김포공항	4	68000	K0002	김태희	F	1982-03-11
Y00398	K0002	P04	명동2	2	38000	K0002	김태희	F	1982-03-11
~~K23212~~	~~K0003~~	~~P05~~	~~강남역1~~	~~1~~	~~12000~~	~~K0003~~	~~유재석~~	~~M~~	~~1971-07-21~~

SQL 구조

SQL Structure : S + F + W

Example 3) *MS ACCESS

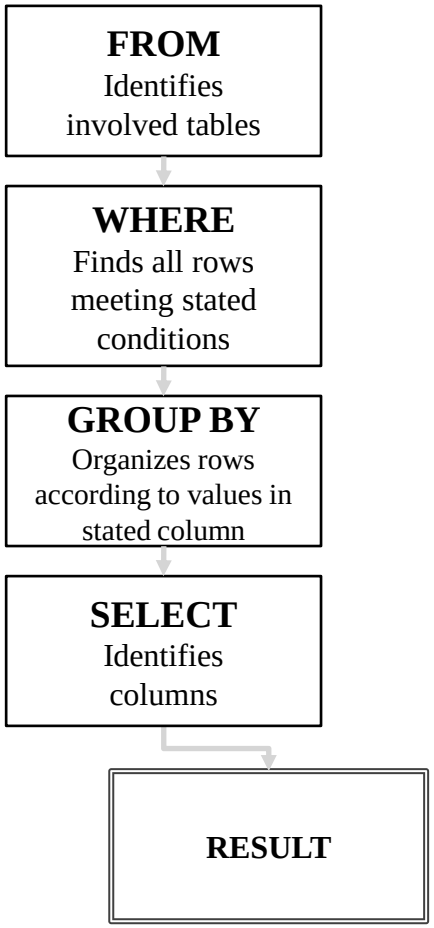


구매ID	Purchase.고객ID	Purchase.상품ID	매장정보	상품개수	구매금액	Customer.고객ID	고객이름	성별	생년월일	Product.상품ID	상품이름	판매가
M10532	K0001	P01	명동1	2	30000	K0001	강호동	M	08-Feb-65	P01	Pasta	15000
D00583	K0001	P02	동대문	1	30000	K0001	강호동	M	08-Feb-65	P02	Steak	30000
K20954	K0001	P01	강남역2	1	15000	K0001	강호동	M	08-Feb-65	P01	Pasta	15000
K30983	K0002	P03	김포공항	4	68000	K0002	김태희	F	11-Mar-82	P03	Sushi	17000
Y00398	K0002	P04	명동2	2	38000	K0002	김태희	F	11-Mar-82	P04	Pizza	19000
K23212	K0003	P05	강남역1	1	12000	K0003	유재석	M	21-Jul-71	P05	Salad	12000

SQL 구조

SQL structure: S + F + W + GB

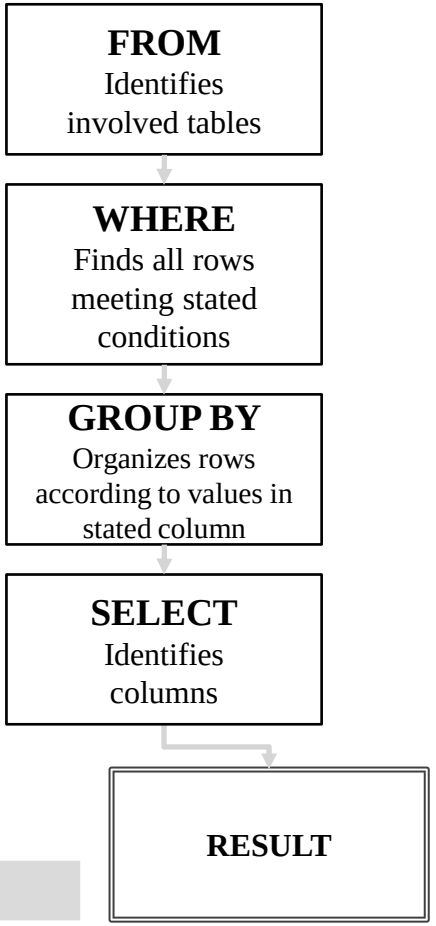
- SELECT <attribute list>
- FROM <table list>
- WHERE <conditions>
- GROUP BY <attribute list>



SQL 구조

SQL structure: S + F + W + GB

- SELECT <attribute list>
- FROM <table list>
- WHERE <conditions>
- GROUP BY <attribute list>



Example 1)

고객ID	sum(구매금액)
K0001	20,000
K0001	30,000
K0001	15,000
K0002	70,000
K0002	35,000

고객ID	sum(구매금액)
K0001	20,000 + 30,000 + 15,000 = **65,000**
K0002	70,000 + 35,000 = **105,000**

SQL 구조

SQL structure: S + F + W + GB

- SELECT <attribute list>
- FROM <table list>
- WHERE <conditions>
- Order BY <attribute list>

Example 1)

DeliveryMethod	PaymentMethod
airplane	Cash
airplane	Cash
airplane	Credit Card
airplane	Credit Card
shipping	Cash
shipping	Cash
shipping	Cash
shipping	Credit Card
shipping	Credit Card
shipping	Credit Card
shipping	Credit Card
shipping	Credit Card

DeliveryMethod	PaymentMethod	Freq
airplane	Cash	2
airplane	Credit Card	2
shipping	Cash	3
shipping	Credit Card	6

FROM
Identifies involved tables

WHERE
Finds all rows meeting stated conditions

GROUP BY
Organizes rows according to values in stated column

SELECT
Identifies columns

RESULT

SQL 구조

SQL structure: S + F + W + GB

Example 2)

구매금액총합/구매횟수

고객이름	구매횟수	구매금액총합	평균구매금액
강호동	3	75000	25000
김태희	2	106000	53000
유재석	1	12000	12000

구매ID	Purchase.고객ID	상품ID	매장정보	상품개수	구매금액	Customer.고객ID	고객이름	성별	생년월일
M10532	K0001	P01	명동1	2	30000	K0001	강호동	M	1965-02-08
D00583	K0001	P02	동대문	1	30000	K0001	강호동	M	1965-02-08
K20954	K0001	P01	강남역2	1	15000	K0001	강호동	M	1965-02-08
K30983	K0002	P03	김포공항	4	68000	K0002	김태희	F	1982-03-11
Y00398	K0002	P04	명동2	2	38000	K0002	김태희	F	1982-03-11
K23212	K0003	P05	강남역1	1	12000	K0003	유재석	M	1971-07-21

SQL 구조

SQL structure: S + F + W + GB

Example 3)

구매금액총합/구매횟수

성별	구매횟수	구매금액총합	평균구매금액
F	2	106000	53000
M	4	87000	21750

구매ID	Purchase.고객ID	상품ID	매장정보	상품개수	구매금액	Customer.고객ID	고객이름	성별	생년월일
M10532	K0001	P01	명동1	2	30000	K0001	강호동	M	1965-02-08
D00583	K0001	P02	동대문	1	30000	K0001	강호동	M	1965-02-08
K20954	K0001	P01	강남역2	1	15000	K0001	강호동	M	1965-02-08
K30983	K0002	P03	김포공항	4	68000	K0002	김태희	F	1982-03-11
Y00398	K0002	P04	명동2	2	38000	K0002	김태희	F	1982-03-11
K23212	K0003	P05	강남역1	1	12000	K0003	유재석	M	1971-07-21

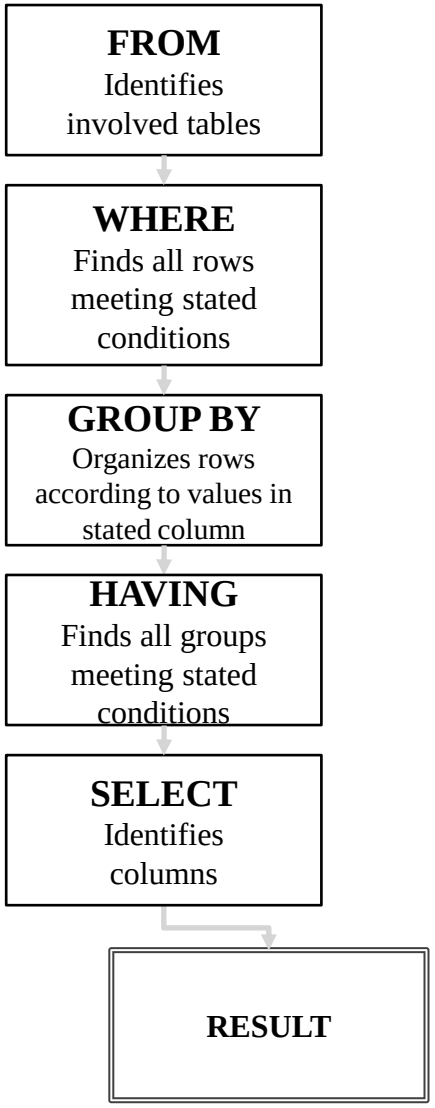
Aggregate Function

- COUNT(): Returns how many rows there are -> How many? How many people? Queries
- SUM(): Returns the total sum -> What is the total sum for the fact? Queries
- AVG(): Returns the average value -> What is the average value for the fact? Queries
- MAX(): Returns the largest value -> What is the maximum value for the fact? Queries
- MIN(): Returns the smallest value -> What is the minimum value for fact? Queries

SQL 구조

SQL structure: S + F + W + GB + H

- SELECT <attribute list>
- FROM <table list>
- WHERE <conditions>
- GROUP BY <attribute list>
- HAVING <conditions>



Example 1)

고객ID	sum(구매금액)
~~K0001~~	~~65,000~~
K0002	**105,000**

고객ID	sum(구매금액)
K0002	**105,000**

SQL 구조

SQL structure: S + F + W + GB + H

Example 2)

HAVING 적용 후

고객이름	구매횟수	구매금액총합	평균구매금액
강호동	3	75000	25000
김태희	2	106000	53000

HAVING 적용 전

고객이름	구매횟수	구매금액총합	평균구매금액
강호동	3	75000	25000
김태희	2	106000	53000
~~유재석~~	~~1~~	~~12000~~	~~12000~~

구매ID	Purchase.고객ID	상품ID	매장정보	상품개수	구매금액	Customer.고객ID	고객이름	성별	생년월일
M10532	K0001	P01	명동1	2	30000	K0001	강호동	M	1965-02-08
D00583	K0001	P02	동대문	1	30000	K0001	강호동	M	1965-02-08
K20954	K0001	P01	강남역2	1	15000	K0001	강호동	M	1965-02-08
K30983	K0002	P03	김포공항	4	68000	K0002	김태희	F	1982-03-11
Y00398	K0002	P04	명동2	2	38000	K0002	김태희	F	1982-03-11
K23212	K0003	P05	강남역1	1	12000	K0003	유재석	M	1971-07-21

SQL 구조

SQL Example – MS ACCESS

Table1 | Query1

ORD_ID	CUST_ID	구매일자	매장	구매금액
1	K001	2009-05-01	명동1	20000
2	K001	2009-07-01	동대문	30000
3	K001	2009-08-03	강남역2	15000
4	K002	2009-04-23	김포공항	70000
5	K002	2009-04-28	명동2	35000

Table1 | Query1

```
SELECT CUST_ID, 구매금액
FROM Table1
WHERE 구매금액 >= 30000
```

Table1 | Query1

CUST_ID	구매금액
K001	30000
K002	70000
K002	35000
*	

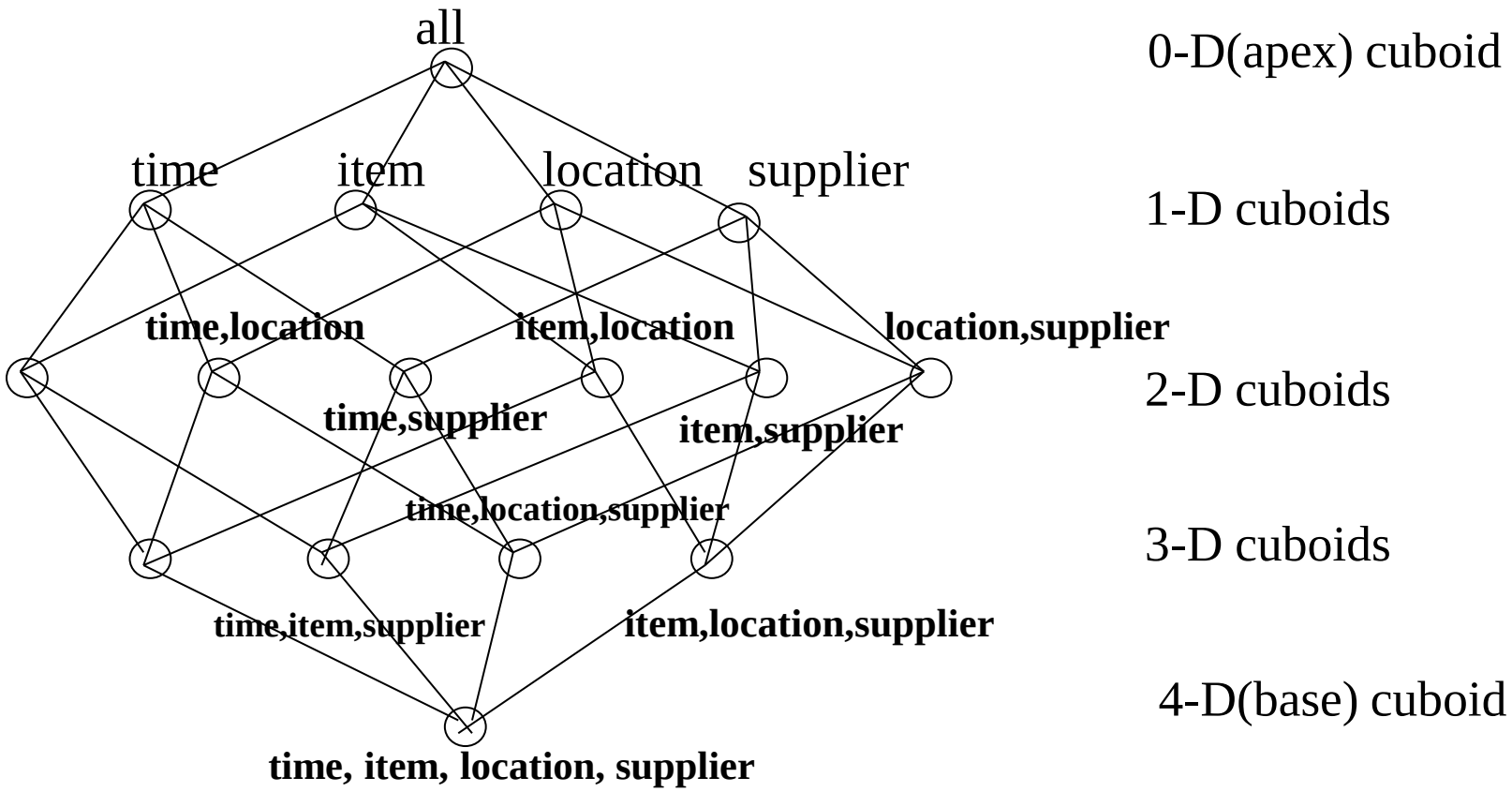
Table1 | Query1

```
SELECT CUST_ID, sum(구매금액)
FROM Table1
GROUP BY CUST_ID
HAVING sum(구매금액) >= 100000
```

Table1 | Query1

CUST_ID	Expr1001
K002	105000

Data Exploration Strategy (OLAP)



0-D(apex) cuboid

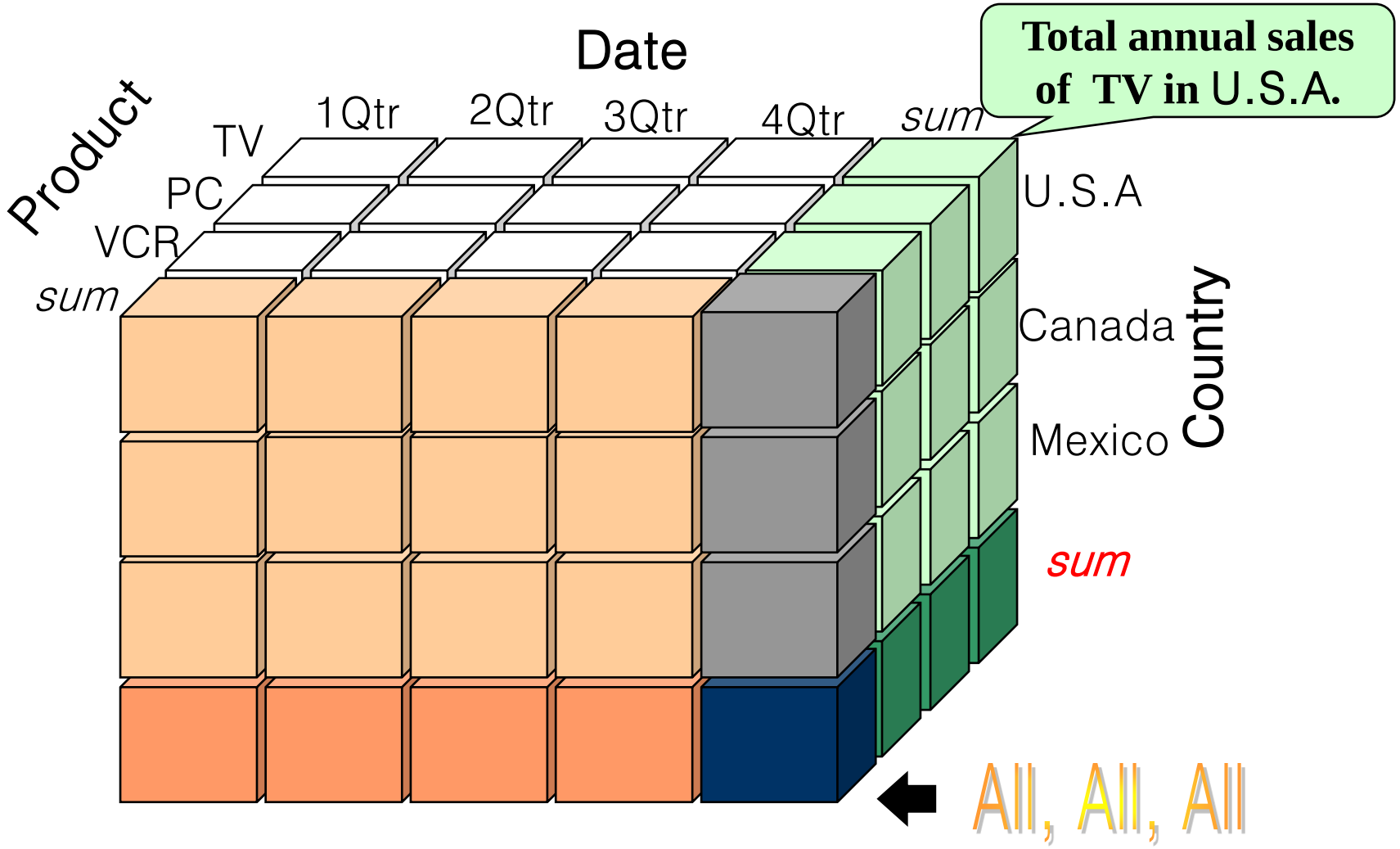
1-D cuboids

2-D cuboids

3-D cuboids

4-D(base) cuboid

Sample Data Cube



Example: Discovery-Driven Data Cubes

item	all
region	all

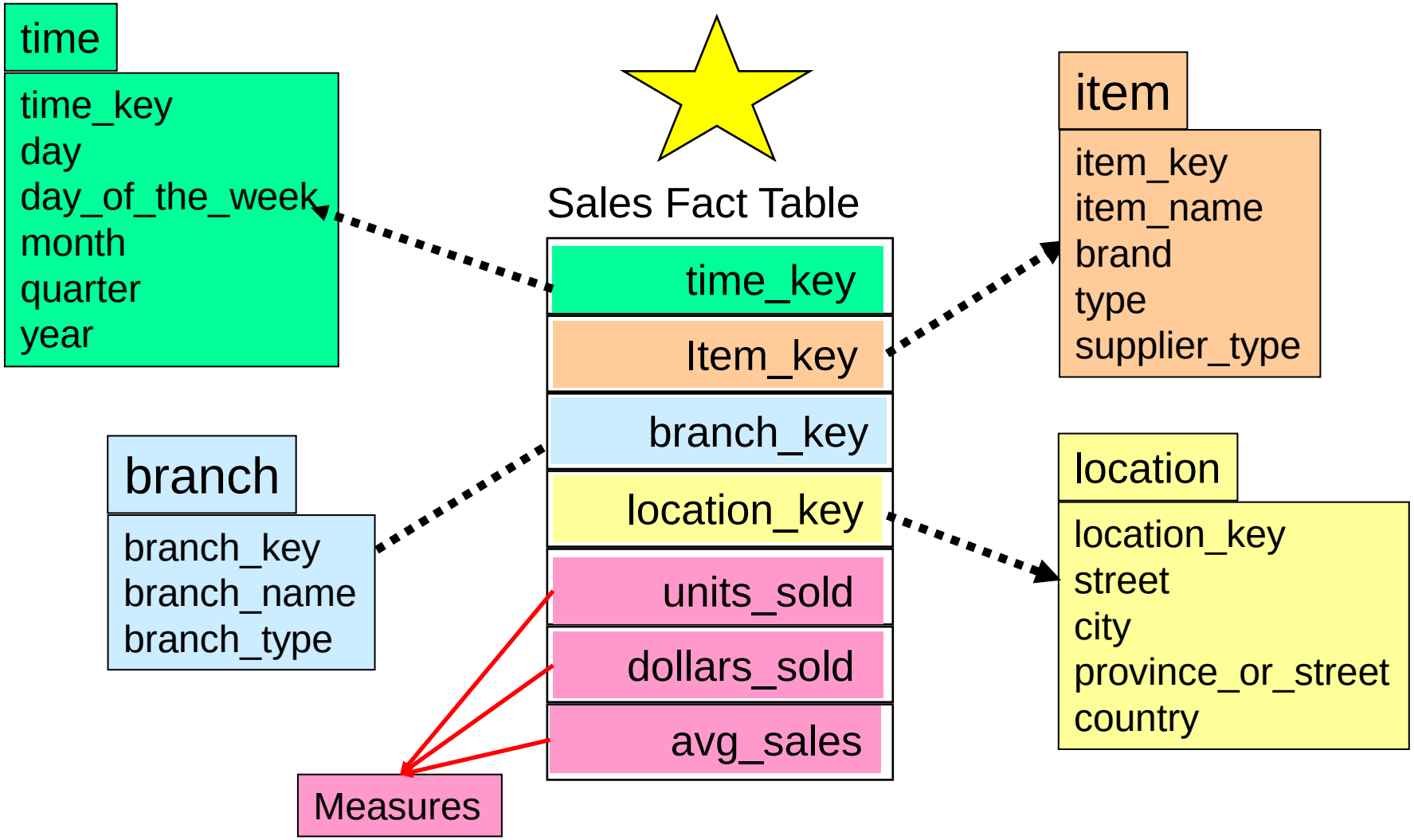
Sum of sales	month											
	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Total		1%	-1%	0%	1%	3%	-1	-9%	-1%	2%	-4%	3%

Avg sales	month											
item	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Sony b/w printer		9%	-8%	2%	-5%	14%	-4%	0%	41%	-13%	-15%	-11%
Sony color printer		0%	0%	3%	2%	4%	-10%	-13%	0%	4%	-6%	4%
HP b/w printer		-2%	1%	2%	3%	8%	0%	-12%	-9%	3%	-3%	6%
HP color printer		0%	0%	-2%	1%	0%	-1%	-7%	-2%	1%	-5%	1%
IBM home computer		1%	-2%	-1%	-1%	3%	3%	-10%	4%	1%	-4%	-1%
IBM laptop computer		0%	0%	-1%	3%	4%	2%	-10%	-2%	0%	-9%	3%
Toshiba home computer		-2%	-5%	1%	1%	-1%	1%	5%	-3%	-5%	-1%	-1%
Toshiba laptop computer		1%	0%	3%	0%	-2%	-2%	-5%	3%	2%	-1%	0%
Logitech mouse		3%	-2%	-1%	0%	4%	6%	-11%	2%	1%	-4%	0%
Ergo-way mouse		0%	0%	2%	3%	1%	-2%	-2%	-5%	0%	-5%	8%

item	IBM home computer

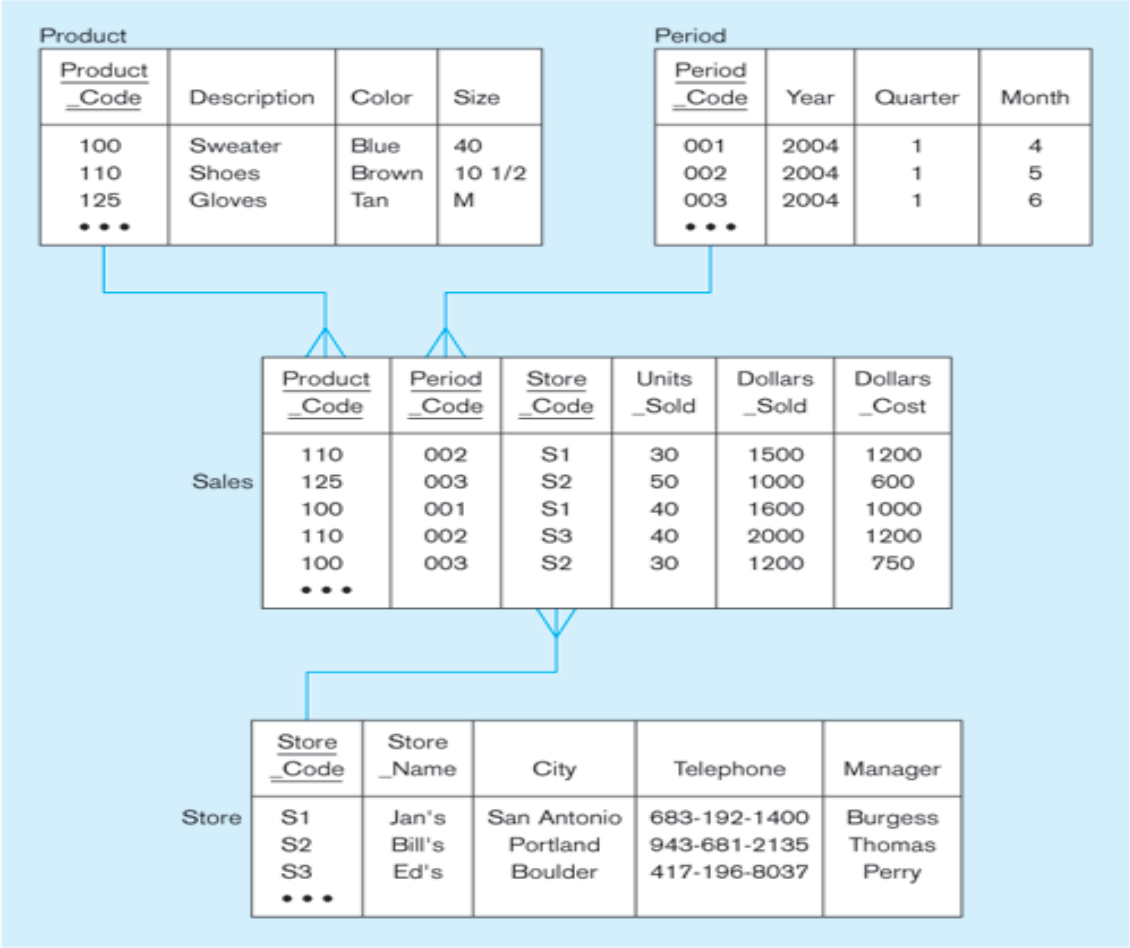
Avg sales	month											
region	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
North		-1%	-3%	-1%	0%	3%	4%	-7%	1%	0%	-3%	-3%
South		-1%	1%	-9%	6%	-1%	-39%	9%	-34%	4%	1%	7%
East		-1%	-2%	2%	-3%	1%	18%	-2%	11%	-3%	-2%	-1%
West		4%	0%	-1%	-3%	5%	1%	-18%	8%	5%	-8%	1%

Star Schema


time
time_key
day
day_of_the_week
month
quarter
year
Sales Fact Table
time_key
Item_key
branch_key
location_key
units_sold
dollars_sold
avg_sales
item
item_key
item_name
brand
type
supplier_type
branch
branch_key
branch_name
branch_type
location
location_key
street
city
province_or_street
country
Measures

Example: Star Schema based data structure

Figure 11-15 Star schema with sample data



Source: Modern Database Management, 11th edition

대화형 질의를 구현하는 SQL

DW and OLAP 에서의 대화형 질의

- 현상에 대해서 궁금한 사항이 발생하기 마련 (Reporting Step)
- 대화형 질의를 통해서 원하는 정보를 얻을 수 있음

- OLAP의 단점은, 전체 큐브의 형태를 정확히 알고 있어야 사용이 수월하다는 것임
- 차원(dimension)과 값(fact or score)이 어떻게 연결되어 있는지 알고 있어야 함

- 관심분야의 Table 구조를 이해하는 것이 모든 것을 알고 있는 것
- 그리고 raw data에서 부터 원하는 데이터를 추출할 수 있는 기술이 필요 (SQL)

Question and Answer

THANK YOU ☺